GLOW 2025: Management & Control

Artur Kraskov
Semester 7
ICT & OL, Delta
Fontys 2025-2026

Content

Introduction

Within the scope of GLOW 2025 Echoes of Tomorrow Interactive Light & Sound installation project a firmware piece was developed. Previous work covered hardware/firmware analysis & advice, design and realization. This document is focused on Management & Control for firmware with GitHub. The objective is to professionalize the development process by establishing version control, release management, teamwork standards, and automated testing.

Objectives

1. Proficient use of Git, branching strategies, and semantic versioning.
2. Automated creation of release artifacts and logs.
3. Facilitating collaboration for future developers.
4. CI pipeline for hardware and software verification.

Background

Analysis & Advice:	https://github.com/Krasnomakov/teensy41_serial_router_firmware/blob/main/docs/GLOW%202025%20Project%20Analysis%20%26%20Advice.pdf

Design:	https://github.com/Krasnomakov/teensy41_serial_router_firmware/blob/main/docs/GLOW%202025_%20Serial%20Router%20Firmware%20Design.pdf
Realization:	https://github.com/Krasnomakov/teensy41_serial_router_firmware/blob/main/docs/GLOW%202025%20Firmware%20%26%20Automated%20test%20realization.pdf
Acceptance test & integration:	https://github.com/Krasnomakov/teensy41_serial_router_firmware/blob/main/docs/GLOW%202025_%20Serial%20Router%20Firmware%20acceptance%20test%20%26%20integration.pdf

Version Management

Objective: Proficient use of Git, branching strategies, and semantic versioning.

Adopted the **GitHub Flow** strategy to ensure a stable `main` branch while allowing flexible feature development.

Repository Structure: Hosted on GitHub with clear separation of production firmware (`actual_code/final_serial_router_protocol_bridge/`) and simulation tools (`host_sim/`).

Branching Strategy:
`main`: Production-ready code.
`feature/*` or `fix/*`: Implementation branches.

Pull Requests (PRs): All changes are merged via PRs to ensure code review and CI checks pass.
Commit Convention: We enforce Conventional Commits (e.g., `feat:`, `fix:`, `docs:`) to facilitate automated changelog generation.
Tagging: Releases are tagged with Semantic Versioning (e.g., `v1.0.0`).

Evidence: See `CONTRIBUTING.md` for the codified rules.

Release Management

Objective: Automated creation of release artifacts and logs.

Implemented a Continuous Delivery (CD) pipeline using GitHub Actions.

Workflow: `.github/workflows/release.yml`
Trigger: The pipeline activates automatically when a tag starting with `v*` is pushed.
Process:

1. Sets up the build environment (PlatformIO).
2. Compiles the firmware for the Teensy 4.1 target.
3. Extracts the binary artifact (`firmware.hex`).
4. Creates a GitHub Release and attaches the binary.

Benefit: Eliminates "works on my machine" build issues and ensures a traceable, reproducible binary for every release.
Evidence: `release.yml` file and GitHub Releases page.

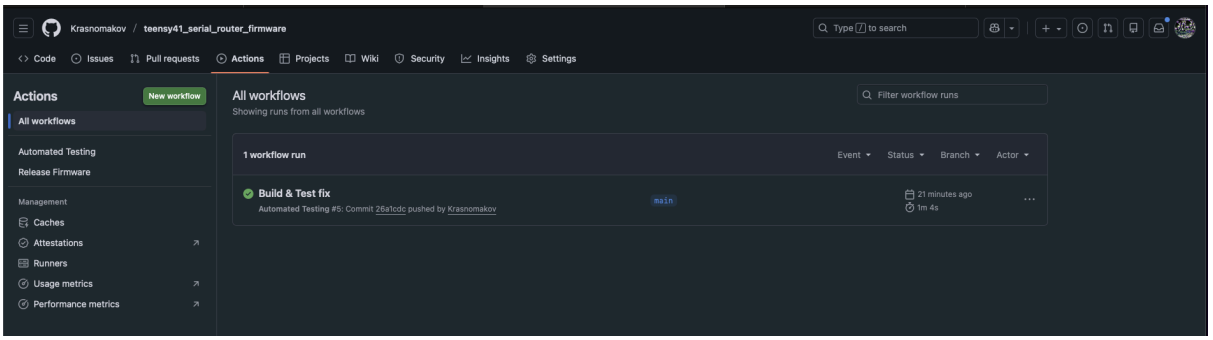

Image 1: Successful GitHub Action run (Build & Test)

Teamwork Support

Objective: Facilitating collaboration for future developers.

To support decreased friction for new team members and external contributors, we standardized the repository structure and Contribution process.

Contribution Guidelines (`CONTRIBUTING.md`): A comprehensive guide detailing how to clone, branch, commit, and submit PRs.

Issue Templates:
Bug Report: Structured form asking for reproduction steps and environment details.
Feature Request: Template to capture user value and implementation ideas.

Pull Request Template: A checklist ensuring self-verification (tests passed, docs updated) before requesting a review.
Code Separation: Refactored the codebase to separate `Teensy` vs `ESP32` firmware sources preventing build conflicts for new developers.

Evidence: `.github/ISSUE_TEMPLATE/`, `CONTRIBUTING.md`.

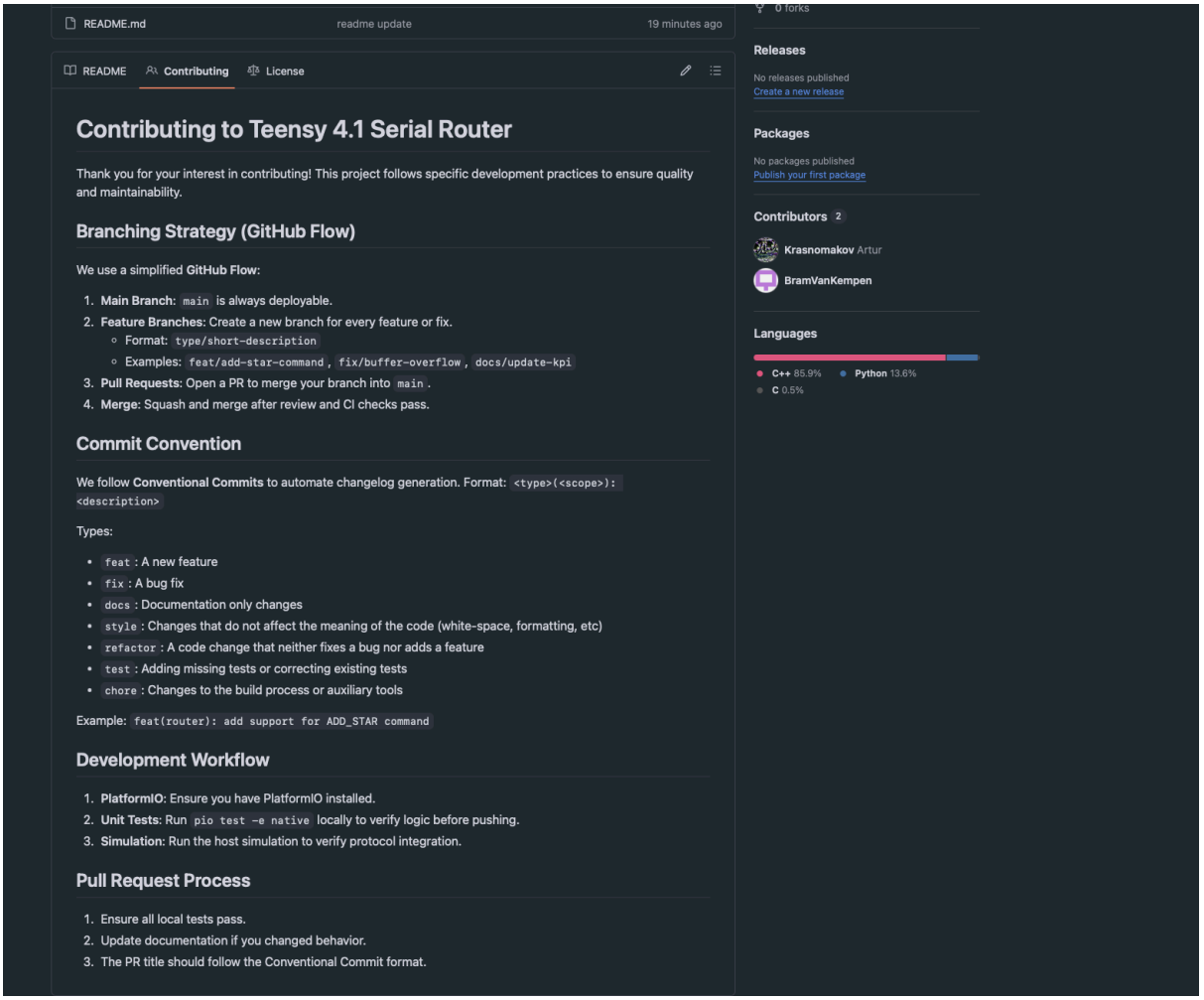

Image 2: Contributing section

Automated Testing

Objective: CI pipeline for hardware and software verification.

Established a Continuous Integration (CI) pipeline to verify both the high-level logic and low-level hardware compilation.

Workflow: `.github/workflows/test.yml`
Trigger: Runs on every `push` and `pull_request` to the `main` branch.

Job 1: Firmware Build & Test (PlatformIO)
Logic Verification (Native): We extracted the protocol parsing logic into a shared library (`actual_code/lib/Protocol`). Using `Unity` and `ArduinoFake`, we run unit tests on the runner (x86) to verify parsing correctness without needing real hardware.

Compilation Check (Teensy 4.1): Runs `pio run -e teensy41` to guarantee the code builds successfully for the target hardware.
Job 2: Simulation Check

Verifies that the accompanying C++ Host Simulation (`host_sim/`) compiles correctly, ensuring the desktop tooling remains functional.

Apple Silicon Support: Documented usage of Rosetta 2 for local mac development to resolve toolchain compatibility issues.

Evidence: `test.yml`, `test_packet_parsing.cpp`, and passing GitHub Action logs.

Conclusion

The repository now fully complies with Manage&Control-H3.1. The infrastructure is automated, enforcing quality standards (Testing) and process discipline (Version/Release Management) without manual intervention, significantly reducing technical debt for the GLOW 2025 project.